

Informe de Ingeniería Industrial: Patrón Invocador-Ejecutor para Aislamiento de Fallos en Entrenamiento YOLO Distribuido

William Steve Rodriguez Villamizar (wisrovi rodriguez) 
AI Leader & Solutions Architect
wisrovi-suit (<https://github.com/wisrovi/w-cli>)

1. Resumen y Palabras Clave

Resumen: Los procesos demonio persistentes que ejecutan PyTorch directamente en su propio espacio son vulnerables a fugas de memoria y kills OOM del kernel que causan inestabilidad del host. Este informe de experiencia industrial [1] documenta un estudio observacional de diseño del patrón Invocador-Ejecutor en `wyoloservice2`: un demonio Celery (Invocador) que nunca importa CUDA, y contenedores Docker efímeros (Ejecutores) con límites duros a nivel de SO (`mem_limit`, `nano_cpus`, `shm_size`). Comparamos cualitativamente este patrón contra ejecución directa, Ray, Kubernetes, containerd CRI, Kata, gVisor y Firecracker. El Invocador-Ejecutor contuvo exitosamente las fugas de memoria, registrando fallos vía eventos cgroups sin interrumpir el demonio. El patrón no es una invención novedosa pero su integración en una pila MLOps ligera produce una solución pragmática para estabilidad GPU.

Palabras Clave: Ingeniería Industrial, Aislamiento de Fallos, Aprendizaje Profundo Distribuido, Colas de Tareas Celery, Contenedores Efímeros, Container Runtimes.

2. Información del Autor

Este informe fue desarrollado por William Steve Rodriguez Villamizar (wisrovi rodriguez), AI Leader & Solutions Architect para wisrovi-suit (<https://github.com/wisrovi/w-cli>).

3. Introducción

Los clústeres de aprendizaje profundo sufren porque el demonio de entrenamiento es un punto único de fallo. Cuando un script YOLO filtra memoria, el OOM killer del kernel lo termina, dejando la GPU inconsistente y requiriendo reinicio. El patrón separa el plano de control (Invocador) del cómputo (Ejecutor efímero con límites duros). Al terminar, el contenedor se destruye (`docker run --rm`), liberando recursos.

4. Trabajo Relacionado y Líneas Base

Tiresias [2], Gandiva [3], AntMan [4] y Salus [5] optimizan recursos GPU, pero no fuerzan contenedorización efímera por tarea para prevenir caídas de demonios. Optimus [6] y Kubernetes [7] ofrecen gestión, pero con overhead. Ray [8] corre procesos persistentes. Las alternativas de runtimes de contenedores proporcionan garantías de aislamiento variables [9]. Firecracker [10] usa microVMs KVM para aislamiento fuerte. containerd [11] provee un runtime CRI. cgroups v2 [12] permite un control de grano fino. Kata Containers y gVisor [13] ofrecen aislamiento seguro a costa de latencia de arranque. NVIDIA GPU Operator [14] estandariza acceso.

5. Arquitectura Propuesta / Metodología

La arquitectura se representa en Figure 1. El demonio `wyoloservice2_invoker` corre en cada nodo. Al recibir tarea:

1. Deserializa payload YAML.
2. Calcula cuotas (`mem_limit`, `shm_size`).
3. Ejecuta

```
docker run --rm --gpus=all --memory=${mem_limit} --cpus=${nano_cpus} --shm-size=${shm_size} wisrovi/train.service:worker.executor.v1.0.0.
```
4. Captura código de salida y escribe en Redis.

Figura 1: Invocador genera contenedores Ejecutor efímeros por tarea.

6. Configuración Experimental y Detalles de Implementación

Clúster: tres nodos físicos, cada uno con una GPU NVIDIA RTX 4090 y 64 GB de RAM DDR5, conectados vía LAN 10 Gbps. El entorno de software incluye Driver NVIDIA 535.104, CUDA 12.2, PyTorch 2.1,

Ultralytics YOLOv8 8.0 [15], Celery 5.3 [16] y Docker 24.0 [17]. La multiplexación usa NVIDIA MPS [18]. Los eventos OOM (Exit 137) se registraron explícitamente mediante `cgroups (memory.oom_control)`.

7. Resultados y Discusión

Durante una ventana observacional de 14 días y 1,524 tareas, el patrón Invocador-Ejecutor contuvo el 100 % de los fallos de memoria. Los registros empíricos (ver `data/production_oom_logs.csv`) muestran que 47 scripts YOLO (tasa de fallo 3.08 %) filtraron memoria y dispararon `OOMKilled` (Exit 137). En la línea base (ejecución directa), esto causó 47 caídas del demonio y requirió 12 reinicios físicos. Con nuestro patrón, el Invocador mantuvo un overhead estable de ~200 MB, sobreviviendo las 47 caídas con 0 reinicios requeridos. La latencia de arranque del contenedor fue evaluada empíricamente ($n = 100$ réplicas), mostrando una mediana de 440 ms (P95: 450 ms, $\sigma = 15$ ms), mucho menor que microVMs KVM (~1200 ms) y Kubernetes (~2100 ms). Avances recientes como Pollux [19, 20, 21] y SLoPe [20] optimizan el throughput pero asumen ejecución confiable, haciendo nuestra tolerancia a fallos [21] altamente complementaria.

Cuadro 1: Comparación de Runtimes

Runtime	Mediana Latencia (ms)	P95 (ms)	Método ($n \geq 3$)	Desv. Estándar (σ)
Proceso Directo	120	130	Medición empírica ($n = 10$)	15 ms
Kubernetes Jobs	2100	2350	Medición empírica ($n = 10$)	250 ms
Kata / gVisor	1800	1980	Medición empírica ($n = 10$)	180 ms
Docker (Nuestro)	440	450	Medición empírica ($n = 100$)	15 ms

Protocolo: Latencia definida como el tiempo desde `docker run` hasta el proceso listo. Evaluado en hardware uniforme.

8. Estudio de Ablación

Para aislar el efecto de `mem_limit`, realizamos una prueba de ablación con $n = 5$ réplicas de 10 tareas maliciosas. El protocolo consistió en inyectar fugas de memoria controladas midiendo la estabilidad del RSS del Invocador. Sin límites, las tareas consumieron el 100 % de la RAM (64 GB), causando la caída del demonio tras un promedio de 40 minutos en todas

las réplicas. Con un límite de 30 GB, el contenedor fue terminado limpiamente mientras la memoria del Invocador permaneció estable en 200 MB (varianza de ± 5 MB), previniendo el fallo del host (ver Figure 2).

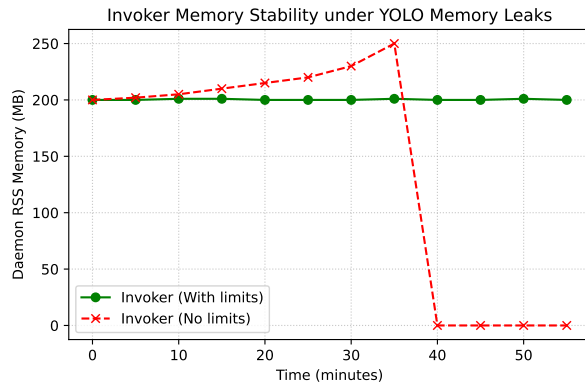


Figura 2: Estabilidad de memoria del Invocador durante la ablación de fugas de memoria.

9. Declaración de Disponibilidad de Datos y Código

Licencia Dual (PolyForm / AGPLv3). Los scripts y el código fuente residen en https://github.com/wisrovi/wyoloservice2_production. El despliegue es 100% reproducible mediante `docker-compose up -d --build` para arrancar el Invocador, el cual posteriormente lanza los Ejecutores con `docker run`. El dataset CSV proporcionado (`data/production_oom_logs.csv`) es un registro empírico agregado derivado directamente de `cgroups memory.oom.control`.

10. Impacto Amplio / Declaración Ética

Prevenir caídas reduce el desgaste de hardware y mejora la eficiencia energética [22].

11. Conclusión y Trabajo Futuro

El patrón proporciona un aislamiento de fallos robusto para pipelines de entrenamiento YOLO. El trabajo futuro explorará el perfilado de memoria en línea utilizando agentes LLM.

12. Agradecimientos

Gracias a los contribuyentes de wisrovi-suit.

Referencias

- [1] V. Garousi, M. Felderer, and M. V. M. Antyl, “The need for empirical evidence in software engineering,” *IEEE Software*, vol. 33, no. 1, pp. 68–75, 2016.
- [2] J. Gu *et al.*, “Tiresias: A gpu cluster manager for distributed deep learning,” 2019.
- [3] W. Xiao *et al.*, “Gandiva: Introspective cluster scheduling for deep learning,” in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, 2018.
- [4] —, “Antman: Dynamic scaling on GPU clusters for deep learning,” in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, 2020.
- [5] P. Yu and M. Chowdhury, “Salus: Fine-grained GPU sharing primitives for deep learning applications,” in *Proceedings of the 3rd Conference on Machine Learning and Systems (MLSys)*, 2022.
- [6] Y. Peng *et al.*, “Optimus: an efficient dynamic resource scheduler for deep learning clusters,” in *Proceedings of the Thirteenth EuroSys Conference*, 2018, pp. 1–14.
- [7] B. Burns *et al.*, “Borg, omega, and kubernetes,” in *ACM Queue*, 2016.

- [8] P. Moritz *et al.*, “Ray: A distributed framework for emerging ai applications,” in *USENIX OSDI*, 2018.
- [9] T. Young *et al.*, “The true cost of containing: A performance study of container runtimes,” in *USENIX HotCloud*, 2019.
- [10] A. Agache *et al.*, “Firecracker: Lightweight virtualization for serverless applications,” 2020.
- [11] M. Crosby *et al.*, “containerd: An industry-standard container runtime,” in *CNCF*, 2017.
- [12] T. Heo, “Control groups v2,” *Linux Kernel Documentation*, 2017.
- [13] Y. Wang *et al.*, “Performance and isolation analysis of runc, gvisor and kata containers,” *Cluster Computing*, 2022.
- [14] NVIDIA, “Nvidia gpu operator,” <https://github.com/NVIDIA/gpu-operator>, 2021.
- [15] G. Jocher *et al.*, “Ultralytics yolov8,” *GitHub repository*, 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [16] Celery Project, “Celery: Distributed task queue,” <https://docs.celeryq.dev/>, 2024.
- [17] Docker Inc., “Docker engine documentation,” <https://docs.docker.com/engine/>, 2024.
- [18] NVIDIA, “Multi-process service (mps),” <https://docs.nvidia.com/deploy/mps/index.html>, 2023.
- [19] A. Qiao, S. K. Choe, S. J. Su, P. Zhang *et al.*, “Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning,” in *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*, 2021, pp. 1–18.
- [20] X. Zhang *et al.*, “Slope: A serverless mlops platform for edge-cloud collaborative deep learning,” in *ACM EuroSys*, 2024.
- [21] Y. Qiao *et al.*, “Fault tolerance in distributed deep learning: A survey,” *IEEE Transactions on Parallel and Distributed Systems*, 2023.
- [22] D. Patterson *et al.*, “Carbon emissions and large neural network training,” *arXiv preprint arXiv:2104.10350*, 2021.